Digital Spectrum Analyzer

1. Veerapot Kitcharoen ID 4718823
2. Tanittha Sutjaritvorakul ID 4718948

This project is a part of the requirement of CE4902,
Computer Engineering Project II

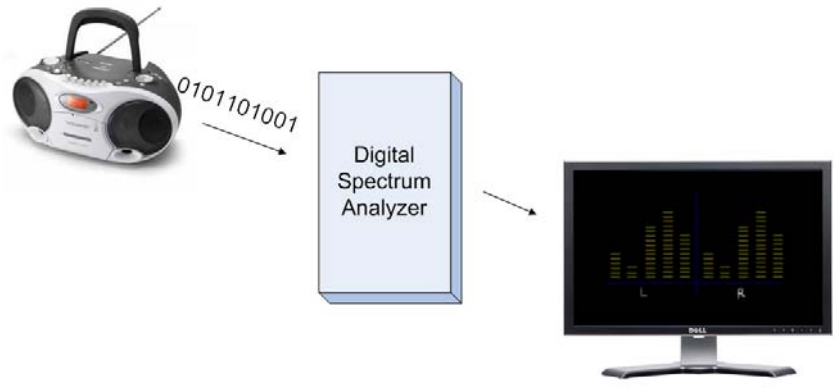
Project Advisor: Dr. Kong Kritayakirana

Department of Computer Engineering

ABAC School of Engineering

Assumption University

Date: 18 / 02 / 2008



Acknowledgments

We would like to acknowledge and extend our heartfelt gratitude to the following persons who have made the completion of this project possible:

Our Project advisor, Dr. Kong Kritayakirana, for his great vital encouragement and support.

Asst. Prof. Dr. Kittiphan Techakittiroj, Chairperson, Department of Mechatronic Engineering,

Dr. Virach Wongpaibool, Department of Telecommunication and Electronics Engineering,

Dr. Jerapong Rojanarowan, Department of Computer and Network Engineering,

Mr. Narong Aphiratsakun, Department of Mechatronic Engineering,

for understanding, assistance and constant reminders.

All Engineering faculty members and Staffs

Most especially to my family and friends.

Abstracts

Hardware engineering is the new field for Thailand since generally focus on software. It is not much improved and also it is rarely to find Hardware engineer in Thailand and companies which can produce or design chips as well.

Nowadays, if we want to produce some equipments which are not required a lot of functionalities or flexibilities, it would be better to be designed by hardware instead of software. Thus we start with this project, *Digital Spectrum Analyzer* which is implemented by pure hardware in order to improve and apply our knowledge in hardware field. After this project is done, we got some benefits over software implementation.

- Size
  With hardware product, you do not have to carry the whole set of computer because the size is much smaller. So it will be convenient as portable equipment.

- Less price and electricity consumption.

Table of contents

Chapter 1: Introduction

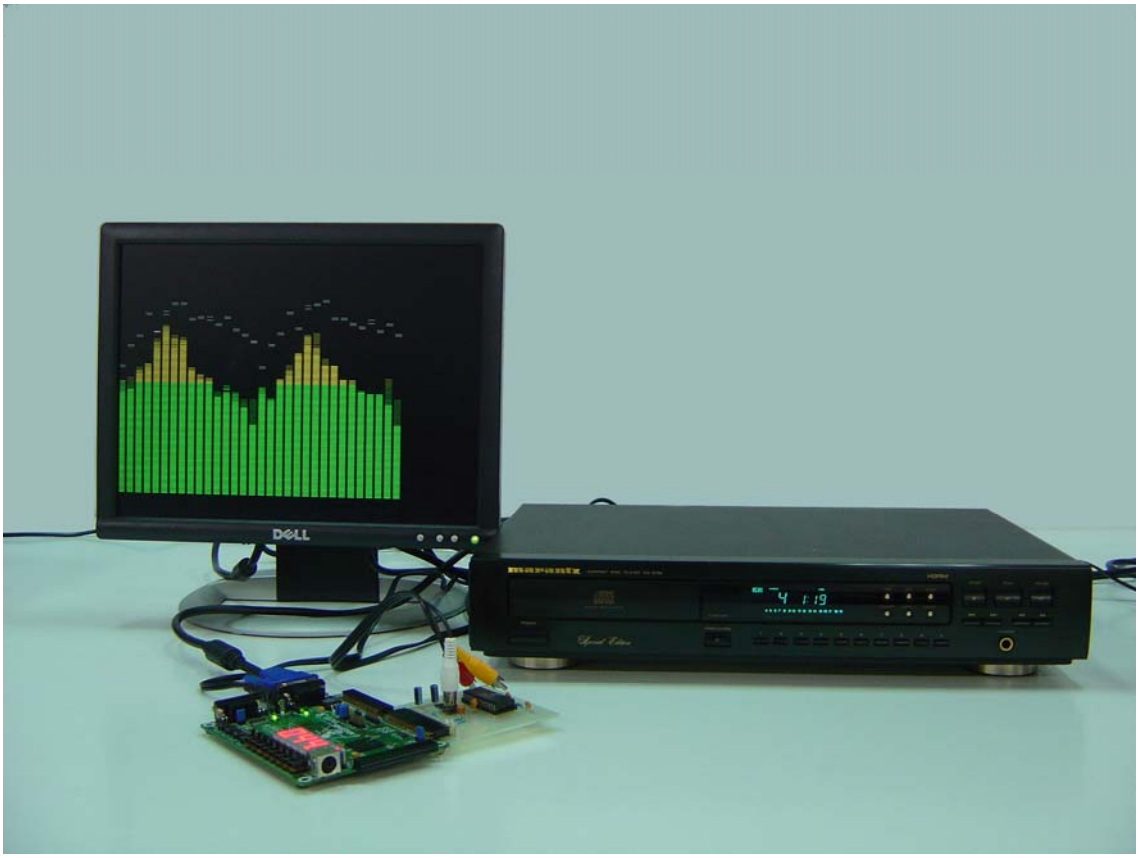
Digital Spectrum Analyzer is an instrument used to display the frequency domain of a digital output plotting amplitude against frequency.

The project consists of:

1. CD Digital Output Player
2. Printed Circuit Board (PCB) – CS8412
3. FPGA
4. VGA monitor

Firstly, the CD player will send the digital data as an input from coaxial port to *Digital Spectrum Analyzer* by getting decoded signal inputs from CS8412 on PCB. CS8412 is used to convert the input to be the usable form then they will be transmitted to FPGA. FPGA will separate the frequency and calculate the amplitude of each frequency level. Then it will transmit the video signals (h, v, r, g and b) to display on VGA monitor.

Regarding to the output on VGA monitor, it is separated in two parts i.e. left and right. There are 32 banks and 16 banks for each side. The height of each bar is different depending on the level of input frequency.



Chapter 2: Theory and Design

Design Guidelines

1. The module name must match the filename and there should have one module per file.
   Reason: It is easy to see which module comes from which file.
   Example: Module name is “xyz” so the filename of module should be “xyz.v”.

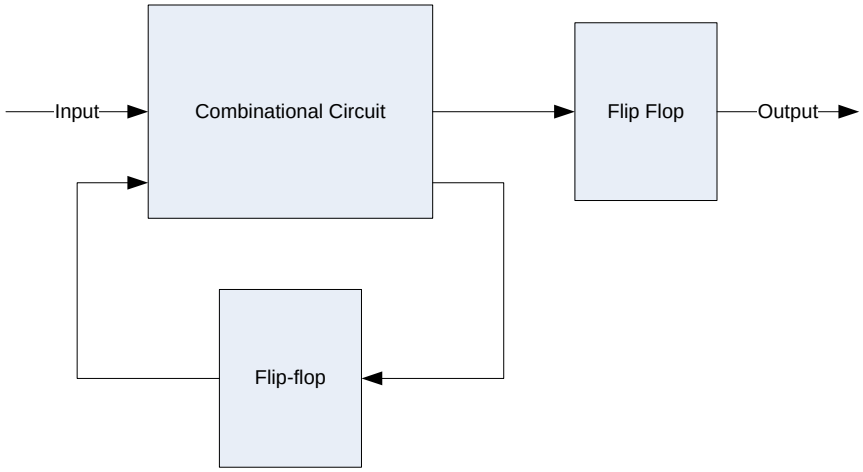
2. Format of signal name: FROM_TO_NAME
   FROM – where does a signal come from?
   TO – where does a signal go to?
   NAME – what is a signal name?
   Reason: To see easily which signal comes from and go to which block.
   Example: At block BB
   Signal XX – AA_BB_XX
   Signal YY – BB_CC_YY

3. Module instantiation must be by explicit port specification.
   Format name of module instantiation (except library):
   MODULE_NAME MODULE_NAME_INSTANCE_NUMBER
   (.PORT_NAME_IN_MODULE_NAME
   (SIGNAL_NAME_IN_THE_MODULE_THAT_INSTANTIATES_THE_MODULE
   _NAME), ……..);
   Example: In “abc.v”, we would like to instantiate “xyz.v”

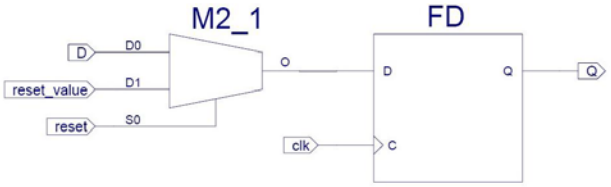
```
xyz  xyz0 (.x(a), .y(b), .z(c));    //this is GOOD example
xyz  xyz0 (    a ,      b ,     c));     //this is BAD example
```

   a, b and c are variables in module “abc.v”
   x, y and z are variables in module “xyz.v”

4. Outputs from every module must be flip-flop output.
   Reason: It is easy to synthesize and debug.
   Example:

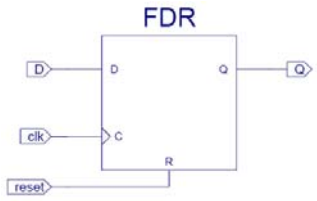


5. The synchronous reset should be used (Active high).



- GOOD

Synchronous reset



- BAD

Asynchronous reset

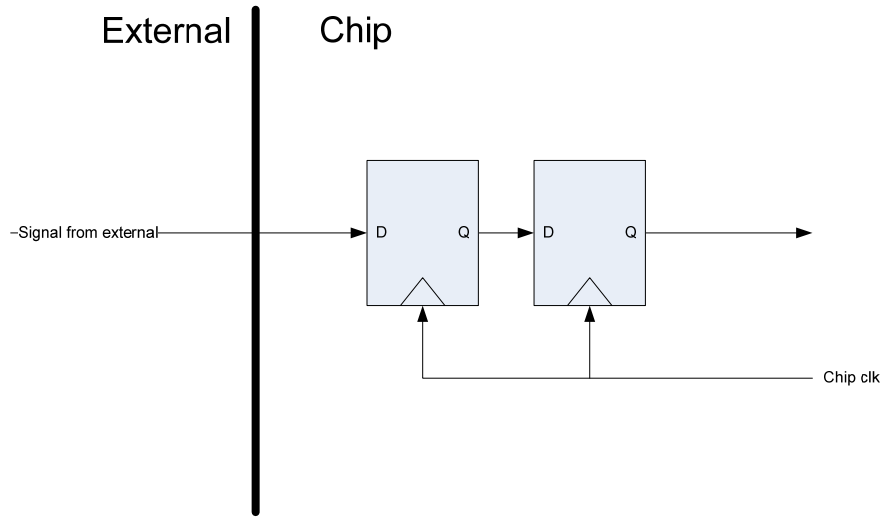
6. Use only active high signals except for compatibility
   Reason: It is easy to code.

7. There should be no implied memory or latch.
   Reason: It is easy to debug.
   Example:

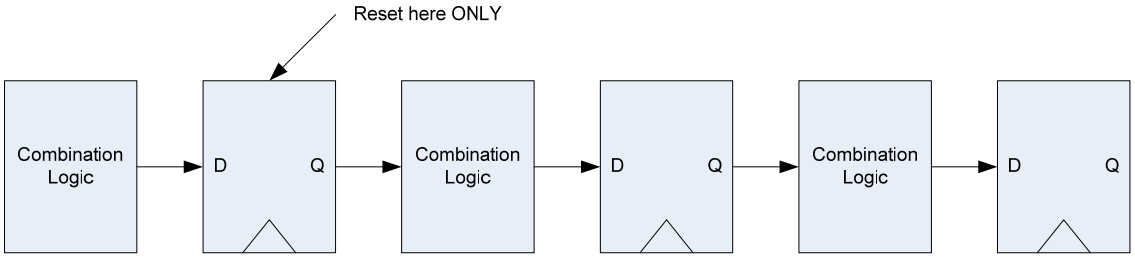
```
always @ ( x or y )        // this is GOOD example.
   s = x;
   if (y)
      s = ~x;
// ===================================
always @ ( x or y )      // this is BAD example.
   if (y)
      s = ~x;
```

8. Synchronizers must be used when signals cross clock domains.
   Reason: It can avoid metastability.

Example:



9. Reset only first flip flop in pipelined design.
   Reason: To guarantee quiescent behavior after reset is released.
   Example:



10. There should not be Bi-Directional signal on chip.
    Reason: To avoid Electromagnetic Interference (EMI).

11. Case statement must be full and parallel.

12. Top module should have instantiation only.

13. A variable should be assigned only one block.
    Reason: To have verilog match actual hardware.
    Example:
    //This is BAD example
    always@(x or y)
    z = -y;
    always@(x or y)
    z = x;
    //==========
    //This is GOOD example
    always@(x or y)

z = x & y;
In second example, you can see that variable z does not appear elsewhere on left hand side.

14. Bus index for n-bit must be indexed as [n-1:0] (zero is the least significant bit).
    Reason: for compatibility with other Xilinx versions.

15. Non blocking assignment should be in Memory block.
    Reason: No signal transition at positive edge of clock except clock itself.
    Example:
    always@(posedge clk)
    a_d1 <= #1 a;

Project Overview

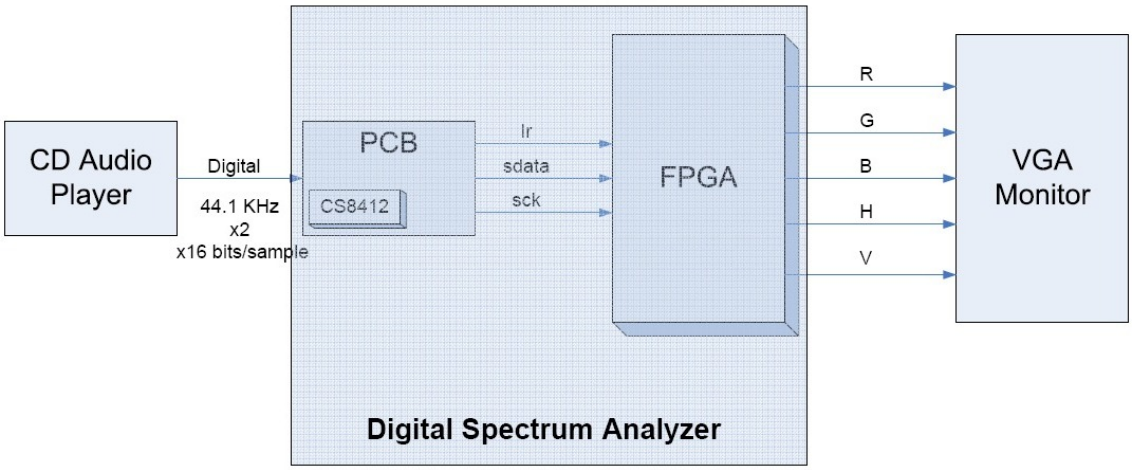
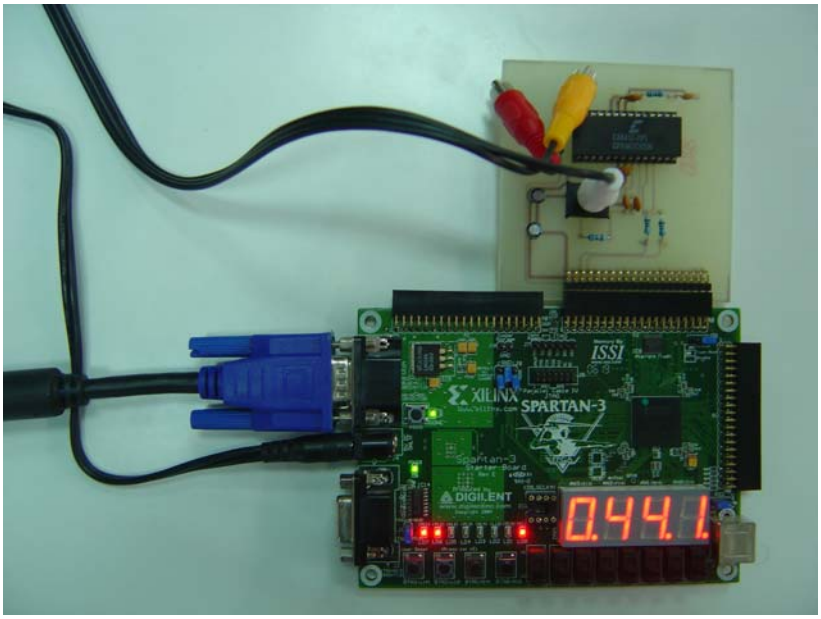


Figure 1 – Project Overview Diagram



Digital Spectrum Analyzer

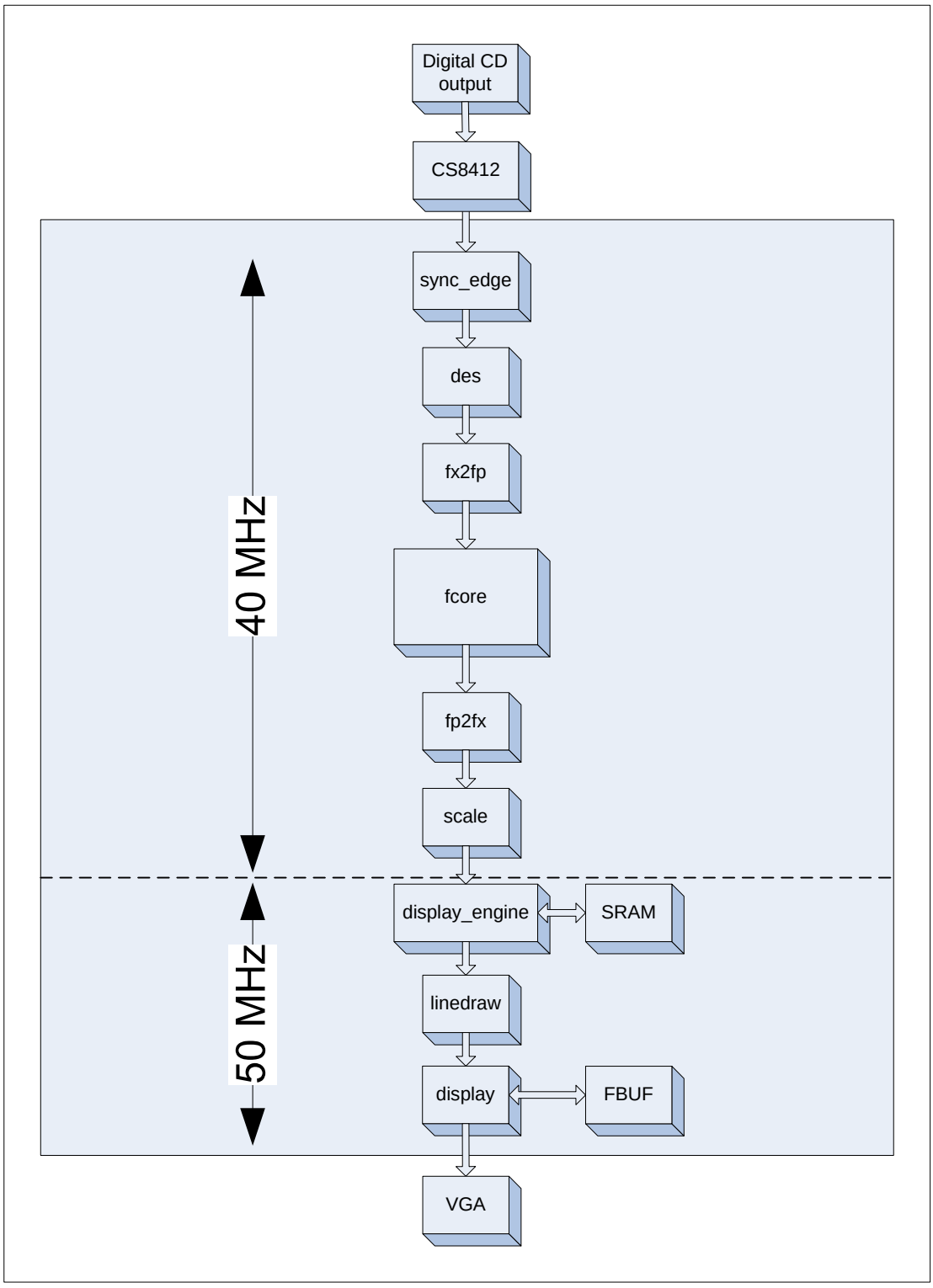


Figure 2 – Digital Spectrum Analyzer

CS8412 Digital Interface Receiver [1]

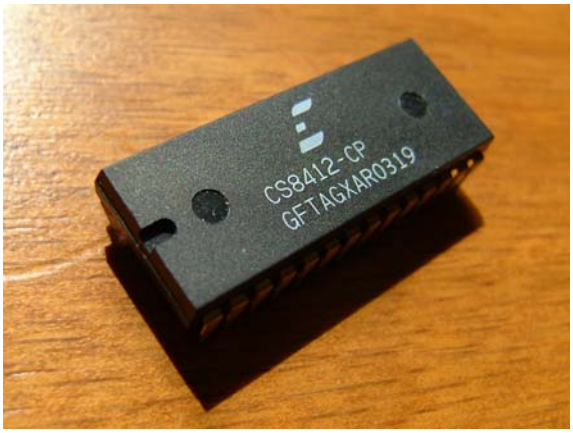


Figure 3 – CS8412 [2]

- A monolithic CMOS circuit that receives and decodes digital audio data which was encoded according to the digital audio interface standards.
- It contains an RS422 line receiver and clock and data recovery utilizing an on-chip phase-locked loop.
- It de-multiplexes the channel, user, and validity data directly to serial output pins with dedicated output pins for the most important channel status bits.

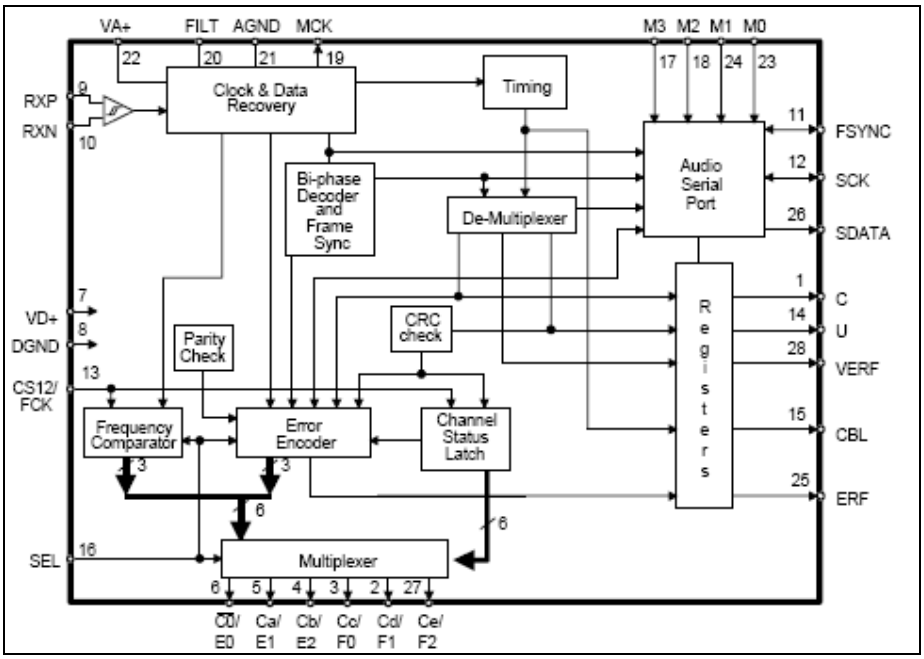


Figure 4 – CS8412 Block Diagram [1]

FPGA (Field-programmable gate array) [3]

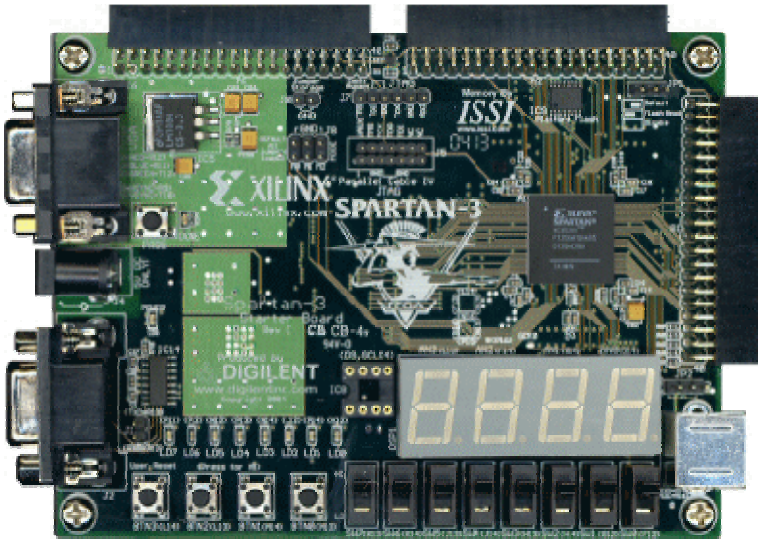


Figure 5 – FPGA [4]

The Spartan-3 Starter Kit board consists of

- 1,000,000-gate Xilinx Spartan-3 XC3S1000 FPGA in a 256-ball Fine-Pitch Thin Ball Grid Array (FTBGA) package (XC3S1000FT256)
  Speed Grade: -4 (standard performance)
  - 17,280 logic cell equivalents
  - Twelve 18K-bit block RAMs (216K bits)
  - Twelve 18x18 hardware multipliers
  - Four Digital Clock Managers (DCMs)
  - Up to 173 user-defined I/O signals
- 2Mbit Xilinx XCF02S Platform Flash, in-system programmable configuration PROM
  - 1Mbit non-volatile data or application code storage available after FPGA configuration
  - Jumper options allow FPGA application to read PROM data or FPGA configuration from other sources
- 1M-byte of Fast Asynchronous SRAM
  - Two 256Kx16 ISSI IS61LV25616AL-10T 10 ns SRAMs

- Configurable memory architecture
    - Single 256Kx32 SRAM array, ideal for MicroBlaze code images
    - Two independent 256Kx16 SRAM arrays
- Individual chip select per device
- Individual byte enables

- 3-bit, 8-color VGA display port
- 9-pin RS-232 Serial Port
    - DB9 9-pin female connector (DCE connector)
    - RS-232 transceiver/level translator
    - Uses straight-through serial cable to connect to computer or workstation serial port
    - Second RS-232 transmit and receive channel available on board test points
- PS/2-style mouse/keyboard port
- Four-character, seven-segment LED display
- Eight slide switches
- Eight individual LED outputs
- Four momentary-contact push button switches
- 50 MHz crystal oscillator clock source
- Socket for an auxiliary crystal oscillator clock source
- FPGA configuration mode selected via jumper settings
- Push button switch to force FPGA reconfiguration (FPGA configuration happens automatically at power-on)
- LED indicates when FPGA is successfully configured
- Three 40-pin expansion connection ports to extend and enhance the Spartan-3 Starter Kit Board
    - FPGA serial configuration interface signals available on the A2 and B1

connectors

- PROG_B, DONE, INIT_B, CCLK, DONE

- JTAG port for low-cost download cable
- Digilent JTAG download/debugging cable connects to PC parallel port
- JTAG download/debug port compatible with the Xilinx Parallel Cable IV and MultiPRO Desktop Tool
- AC power adapter input for included international unregulated +5V power supply
- Power-on indicator LED
- On-board 3.3V , 2.5V , and 1.2V regulators

VGA

Figure 6 – VGA Monitor [5]

The output will be shown on VGA monitor and the specification will be shown as following.
- Refresh rate = 60Hz
- 512 x 480 pixels
- 8 colors which are black, blue, green, cyan, red, magenta, red and white.

Design

sync_edge

There are three signals which got from CS8412.

- ext_sync_edge_sck – Serial clock
  Generate 32 clocks for every audio sample.
- ext_sync_edge_sdata – Serial Data
  - Audio data serial output.
  - Can be changed only at positive edge of clock of serial clock.
- ext_sync_edge_lr – Frame Sync
  - Delineate the serial data and indicate the particular channel, left or right.
  - ext_sync_edge_lr = 1 (left), ext_sync_edge_lr = 0 (right)
  - Can be changed only at positive edge of clock of serial clock.

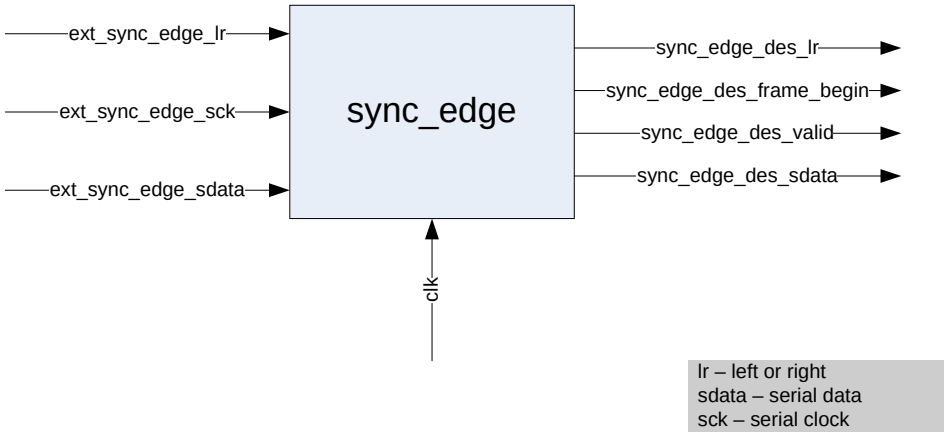


Figure 7 – *sync_edge*

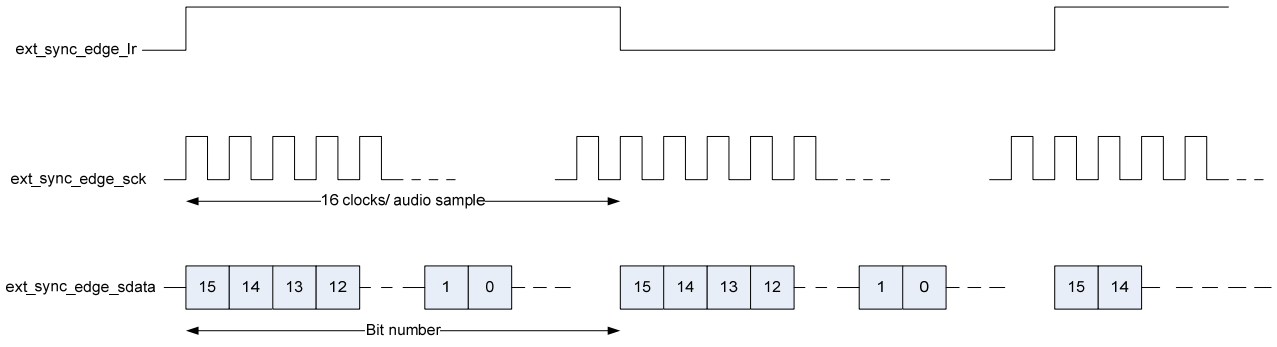
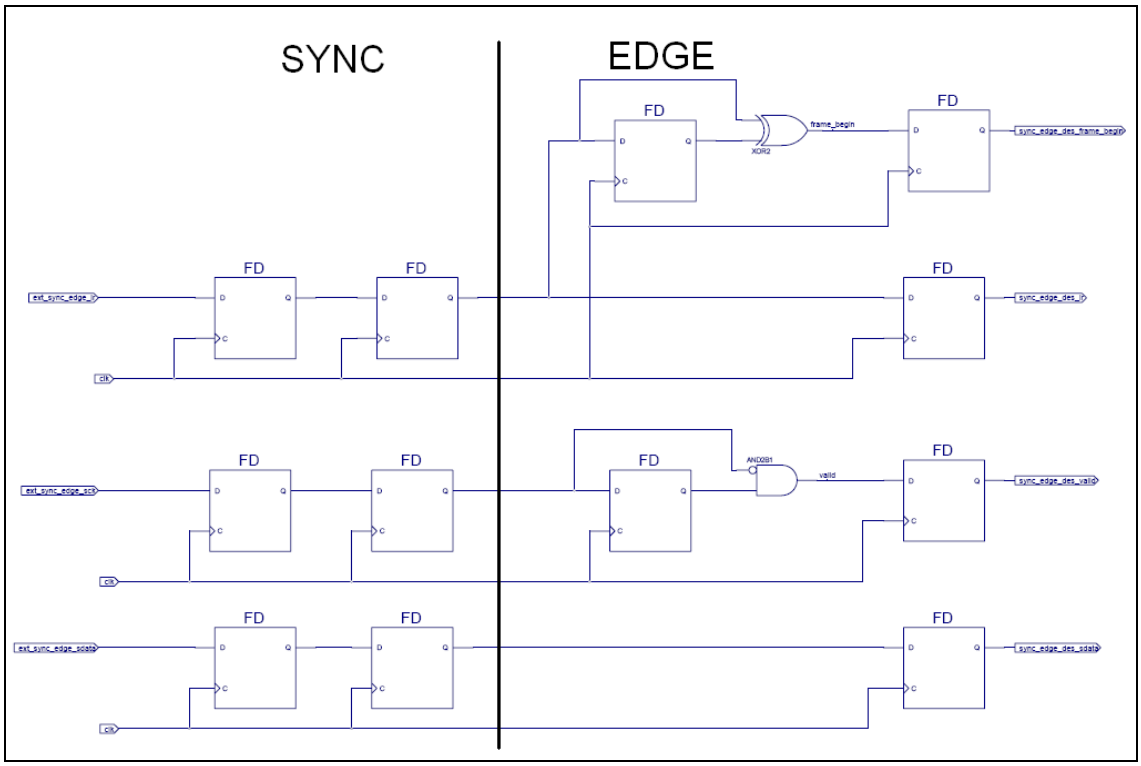


Figure 8 - functional specification of *ext_sync_edge*

In *sync_edge*, we can divide roughly in two parts i.e. sync (synchronize) and edge.

1. sync – according to design guidelines, all signals need to be synchronized when they cross clock domains to avoid metastability. So those signals which come from CS8412 will be passed through two flip-flops.

2. edge – this part has an edge detector to detect the beginning(frame_begin) and the validity(valid) of serial data.



des (deserializer)

There are five signals which got from *sync_edge*.

- sync_edge_des_lr
  - Delineate the serial data and indicate the particular channel, left or right.
  - sync_edge_des_lr = 1 (left), sync_edge_des_lr = 0 (right)
  - Can be changed only at positive edge of clock of serial clock.
- sync_edge_des_sdata
  - Audio data serial output.
  - Can be changed only at positive edge of clock of serial clock.
- sync_edge_des_frame_begin
  - This signal shows the beginning or most significant bit of serial data regarding to frame sync.
  - sync_edge_des_frame_begin is asserted for one clock.
  - It can be asserted only when sync_edge_des_lr changes from 1 to 0 or 0 to 1.
- sync_edge_des_valid
  This signal shows the validity of serial data regarding to serial clock.
  - sync_edge_des_valid is asserted for one clock.
  - It will be valid (sync_edge_des_valid = 1) only when serial clock (sync_edge_des_sck) is changed to be high or 1.

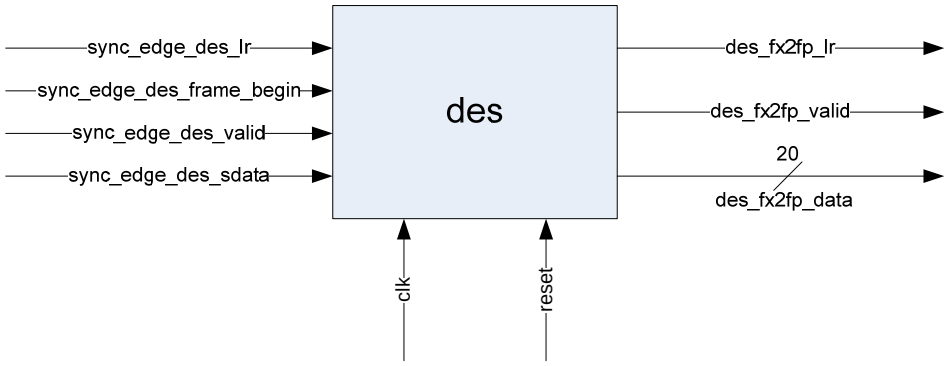


Figure 9 – *des*

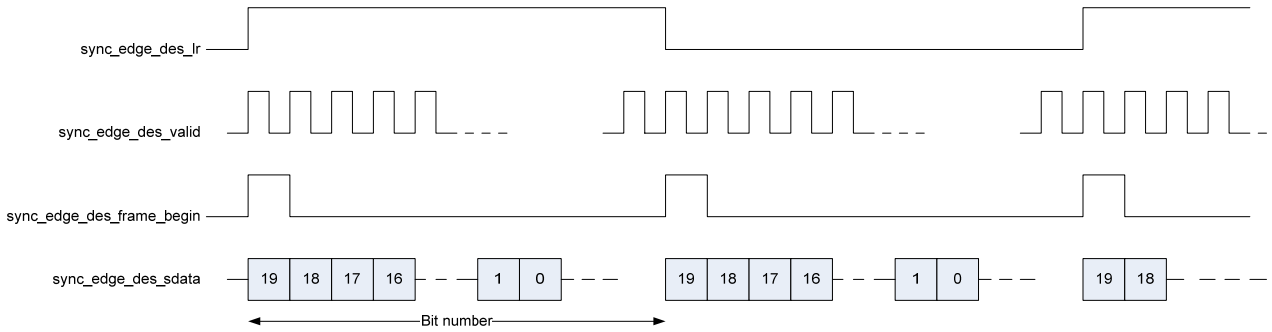


Figure 10 – functional specification of *sync_edge_des*

In *des*, it has a finite state machine to collect 20 bits from 1-bit serial data and pack into a 20-bits data. Then valid signal will be generated if a 20-bits data packing is done.

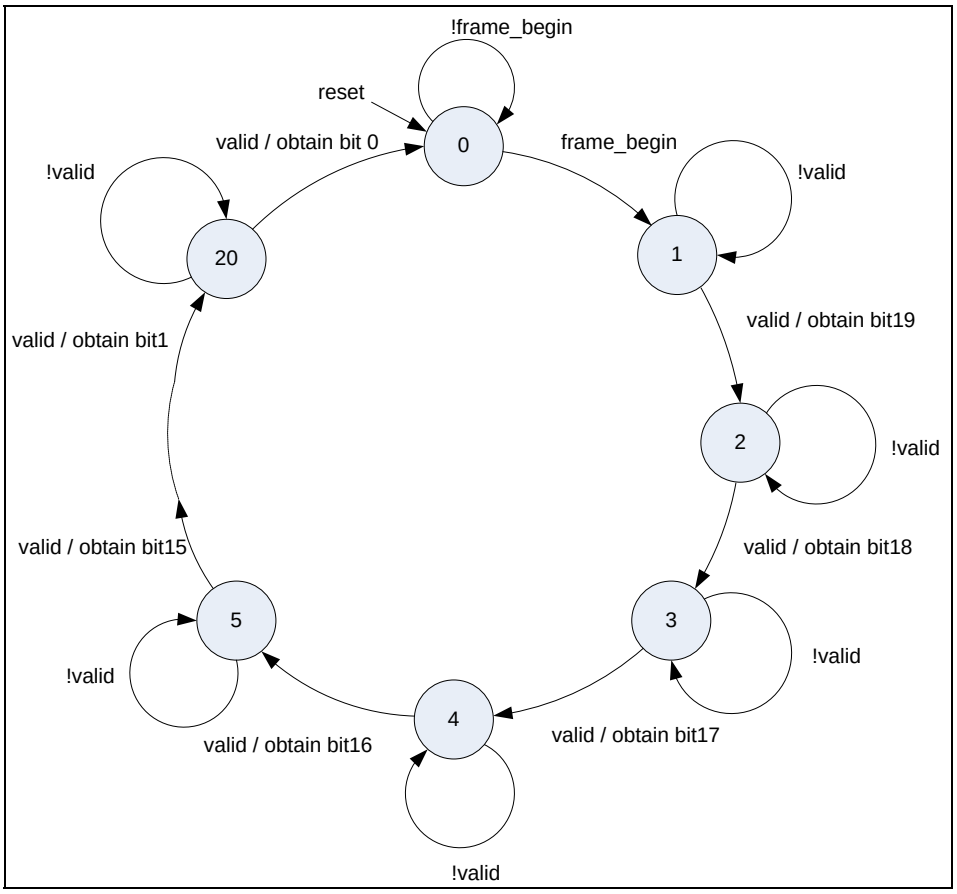


Figure 11 - Finite State Machine of *des*

fx2fp (fixed point to floating point)

There are three signals which got from *des*.

- des_fx2fp_lr
    - Delineate the serial data and indicate the particular channel, left or right.
    - des_fx2fp_lr = 1 (left), des_fx2fp_lr = 0 (right)
    - Can be changed only at positive edge of clock of serial clock.
- des_fx2fp_valid
  This signal will show the validity of 20-bits data.
- des_fx2fp_data
  Audio data output with 20 bits.

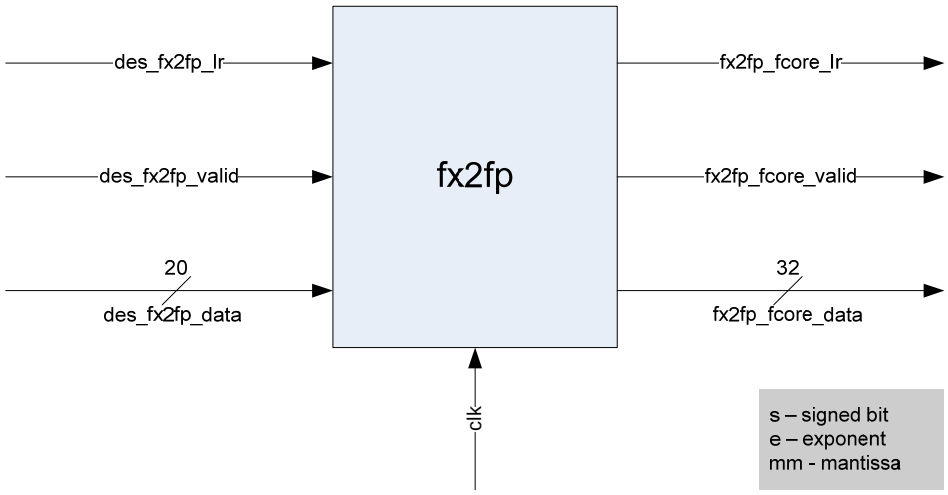


Figure 12 – fx2fp

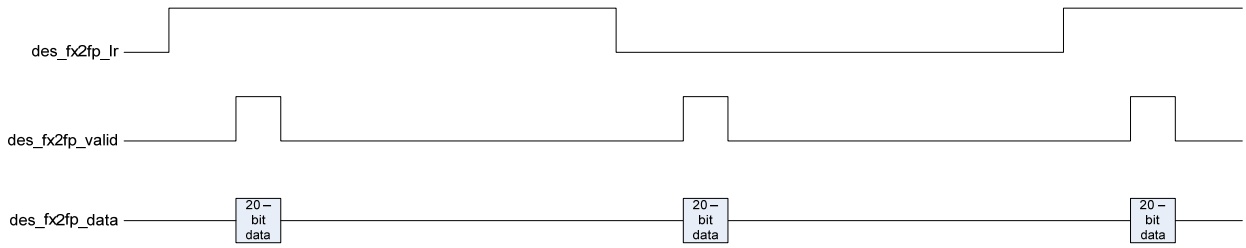


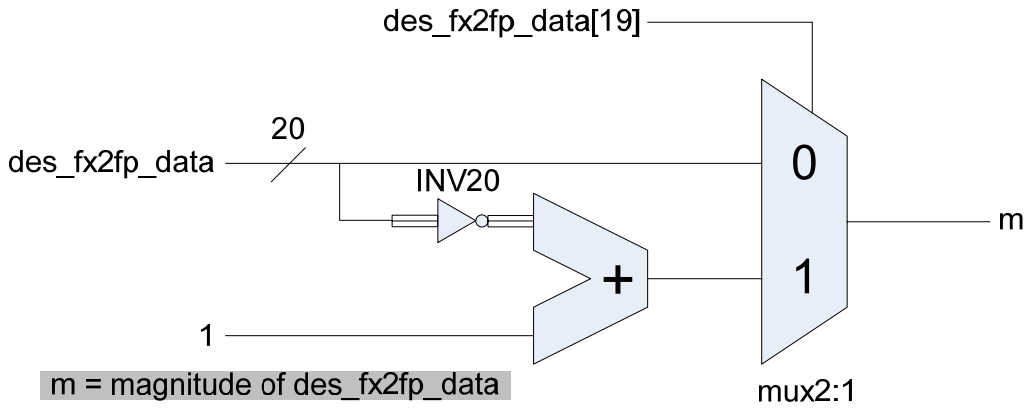
Figure 13 – functional specification of *des_fx2fp*

In *fx2fp*, it converts fixed point to floating point since fixed point does not have wide dynamic range. *fx2fp* will get the magnitude from the data then look for the first 1's in des_fx2fp_data's magnitude starting from the most significant bit then it will generate exponent and mantissa by using a priority encoder as the following table.

Fixed point format: signed fixed point numbers in two's complement

Floating point format:

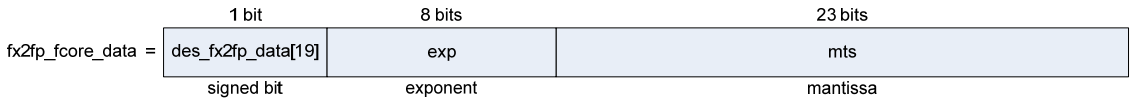
Getting magnitude from des_fx2fp_data



Magnitude of des_fx2fp_data (20 bits) **m**	**Bit number of data that found first 1's (des_fx2fp_data[19])**	**8 – bit Exponent (Decimal format) - exp**	**23 – bit Mantissa** **mts**
1xxxx_xxxxx_xxxxx_xxxxx	19	127	{m[18:0], 4'b0}
01xxx_xxxxx_xxxxx_xxxxx	18	126	{m[17:0], 5'b0}
001xx_xxxxx_xxxxx_xxxxx	17	125	{m[16:0], 6'b0}
……	……	……	……
00000_00000_00000_001xx	02	107	{m[1], 21'b0}
00000_00000_00000_0001x	01	109	{m[0], 22'b0}
00000_00000_00000_00001	00	108	23'b0
00000_00000_00000_00000	-	0	23'b0

Note: x = don't care

After getting the exponent and mantissa then it will connect them together and the MSB of the fx2fp_fcore_data[19] which is signed bit regarding to the floating point format.



fcore

There are six signals which got from *fx2fp*.

- fx2fp_fcore_lr
  Delineate the serial data and indicate the particular channel, left or right.
- fx2fp_fcore_valid
  This signal will show the validity of 32-bits floating point data.
- fx2fp_fcore_data – floating point which has 32 bits.

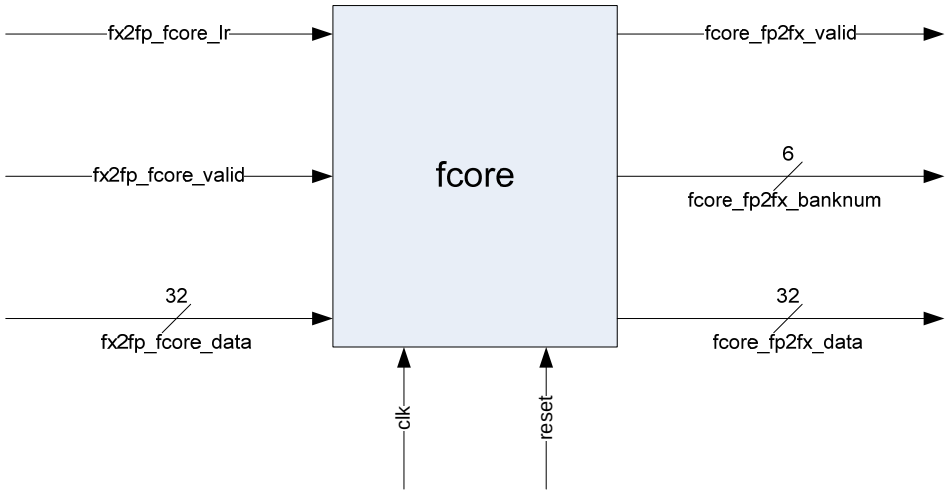


Figure 144 – fcore

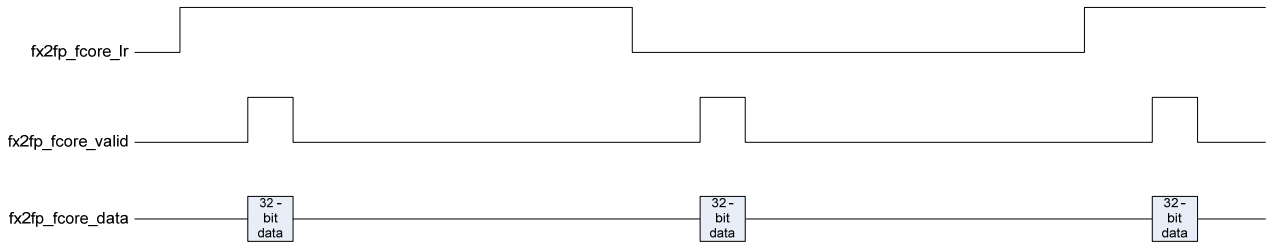


Figure 155 – functional specification of fx2fp_fcore

In *fcore*, it generates bank number according to the data. There are the assembly instructions come from *fcore* e.g. adding, multiplication, etc. totally 25 instructions. Each of instruction uses one clock cycle. Hence, it can go around 40MHz.

2 channels x 16 banks/channel x (25 assembly instructions/filter bank) per bank x 44100 samples/second = 35.28MHz ≈ **40MHz**

fp2fx (floating point to fixed point)

There are three signals which got from *fcore*.

- fcore_fp2fx_valid
  This signal will show the validity of 32 bit - floating point data.
- fcore_fp2fx_banknum
  Generate the number of bank according to the frequency domain.
- fcore_fp2fx_data – floating point with 32 bits.

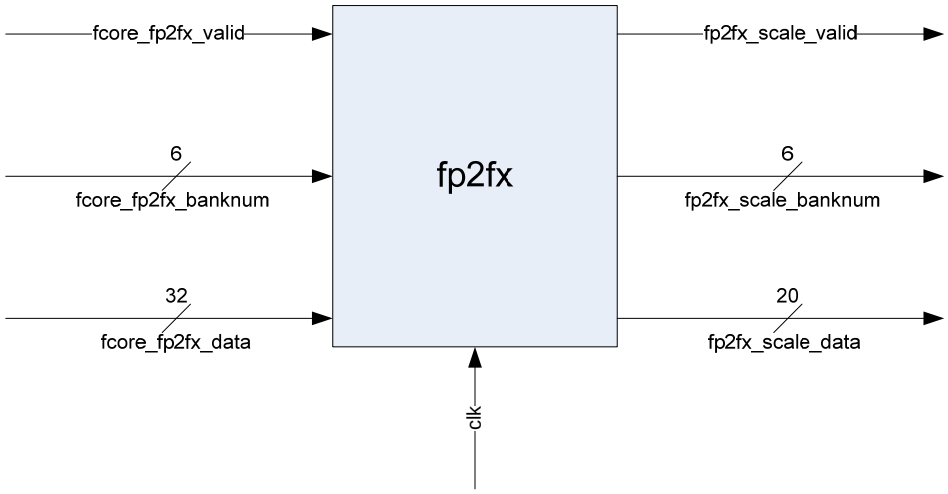


Figure 166 – *fp2fx*

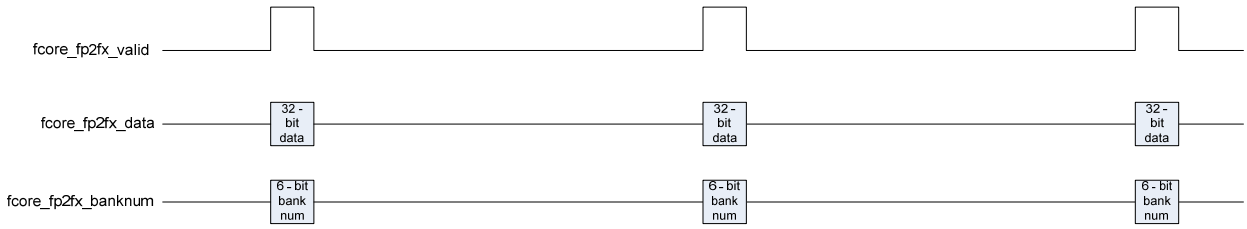
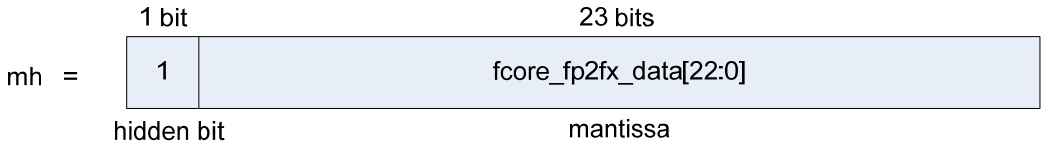


Figure 17 - functional specification of *fcore_fp2fx*

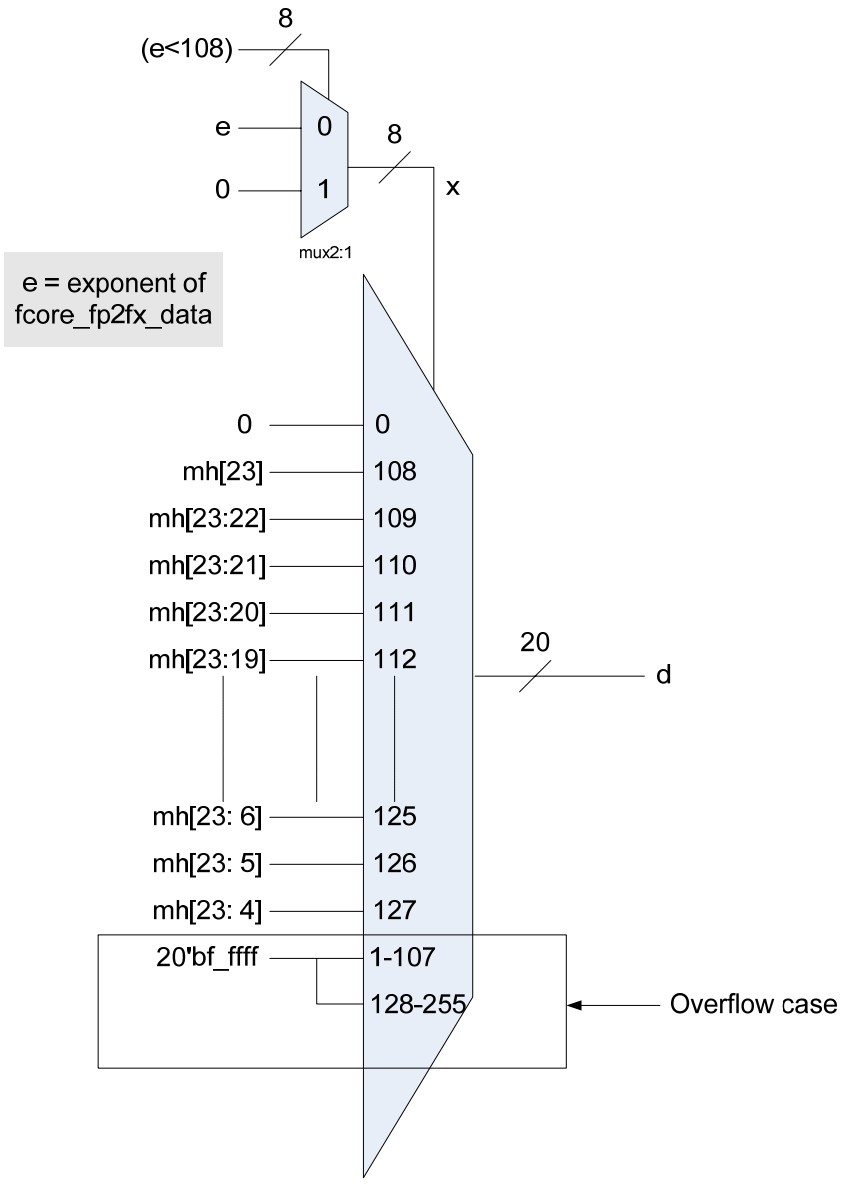
In *fp2fx*, it converts floating point that has been used in *fcore* back into fixed point.

1. It separates each part of floating point from fcore_fp2fx_data i.e. sign, exponent, mantissa.

2. It puts the hidden bit in front of mantissa of fcore_fp2fx_data.



3. It will generate the new data of fixed point format according to the exponent and mh as following schematic.



4. After getting the new data then it will put the sign bit (fcore_fp2fx_data[31]) as the most significant bit of the new data and concatenate with d exclude MSB of d.

scale

There are three signals which got from *fp2fx*.

- fp2fx_scale_valid
  This signal shows the validity of fixed point data.
- fcore_scale_banknum
  The value of bank number according to the frequency domain.
- fcore_scale_data
  The audio output which is in fixed point format since it is converted by *fp2fx*.

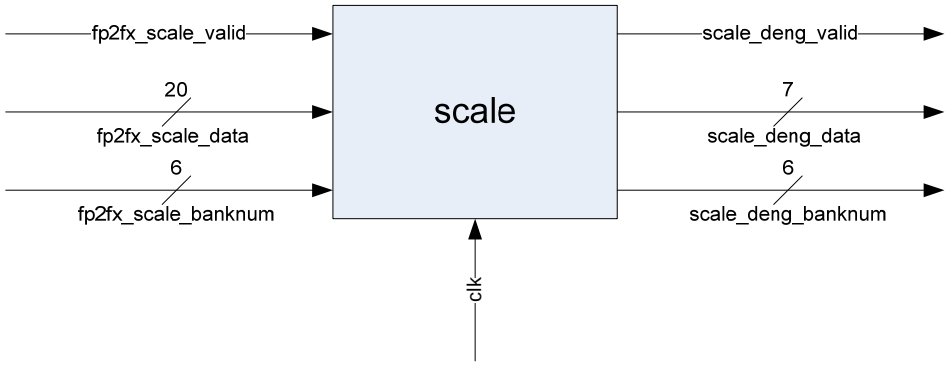


Figure 18 – *scale*

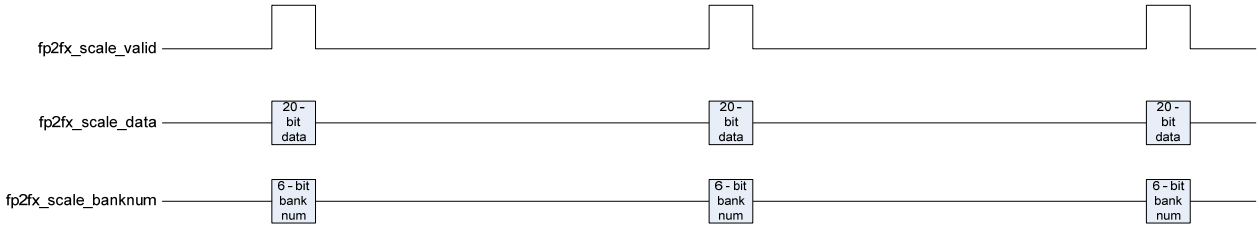


Figure 19 – functional specification of *fp2fx_scale*

In *scale*, it generates the new value of data from 20 bits to be 7 bits and makes it be proper value. The data from *fx2fp* goes into logarithmic lookup table since the range of human hearing is quite large so the decibel unit is applied as log scale.

deng (display engine)

There are three signals which got from *scale*.

- scale_deng_valid – validity of scale_deng_data
- scale_deng_data – 7-bit data which is rescaled from *scale*
- scale_deng_banknum – the bank number of the data

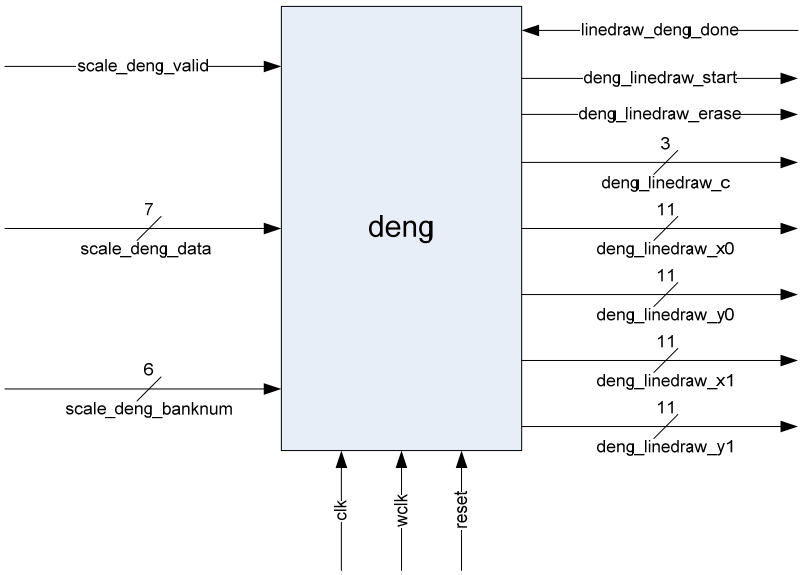


Figure 20 – *deng*

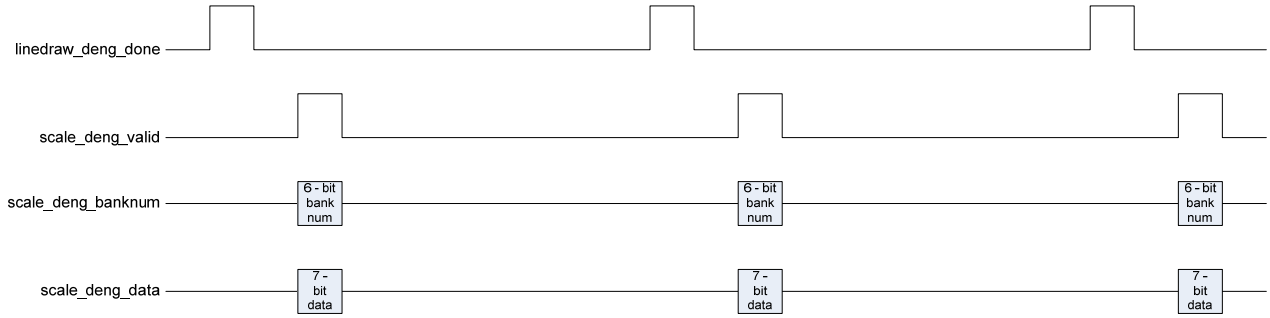


Figure 21 – functional specification of *scale_deng*

In *deng*, it has a finite state machine to generate each point to draw the line in x axis and y axis. It will start to send the value line by line of each bank. Before generating the coordinates, all inputs have to go to SRAM as we have two different frequencies i.e. write clock = 40MHz(wclk) and read clock = 50MHz(clk) since *fcore* cannot perform in 50MHz and the calculation for timing display will be easier as well because the pixel clock of VGA timing control is 25MHz (50 is the double of 25). Here is algorithm of *deng*

```
// This is algorithm of deng (display engine) 2008 02 22
// This display engine can generate horizontal line only
// b = current bank number
// v = current vertical line which will be drawn in current bank number
// value[b] = the new data in that current bank number
// erase = the erase signal will remove the surplus of previous vertical line if the new data is
less than the previous one.
// BMAX, VMAX, LEN_X and GAP_Y are constant values
// BMAX = max bank number (32 banks)
// VMAX = max vertical line (128 vertical lines)
// LEN_X = length of x (10 pixels)
// GAP_Y = gap between each vertical line (3 pixels)
// (x0, y0) = initial coordinate
// (x1, y1) = final coordinate
// inv_v = inverse value of v
// tout = time out counter
// cout = color of the line
// start = this signal will be asserted when the coordinates are generated
// The display resolution is 512x480
// The initial coordinate will start at the TOPLEFT corner of the screen
// + -- -- x -- -->
// | ********************
// | *                  *
// y *                  *
// | *                  *
// | *                  *
// V *                  *
//   ********************
// For the color, we use 3 bit-color i.e. RGB - red, green, blue.
// MSB of color is red and LSB of color is blue.
// This is only for one screen.
// After deng finished giving the value for the whole screen, it will reset then start from the
beginning with b=0 and v=0

for (b=0; b<=BMAX; b=b+1){
    for (v=0; v<=VMAX; v=v+1){
    if (value[b]<v)
                erase=1;
             else
                erase=0;
//=================================================
// This part will generate the value of x0, y0, x1, y1 for linedraw
// ====== Horizontal ====== x0 and x1 = fn(b)
// horizontal display resolution is 512 pixels, 32 banks
// There will be 512/32 = 16 pixels/bank and shift right by 2 pixels
// the length of x is 10 pixel so the gap between each bank is 16 - 10 = 6 pixels
        x0 = b*16 + 2;
        x1 = x0 + LEN_X;
// ====== Vertical ======== y0 and y1 = fn(v)
// vertical display resolution is 480 pixels, 128 lines
// The gap between each line is 3 pixels and shift down by 50 pixels
        inv_v = ~v;
        y0 = (inv_v*GAP_Y)+ 50;
         y1 = y0;
//=================================================
// If deng gets signal 'done' from linedraw or time is out,
// it can generate the next value for linedraw by starting with the next bank
        tout = 0; start = 1;
        while (!done || tout <= 2000){
        tout = tout +1;
        }
    }
}
// This part we specify the color range of vertical line and the color to erase
if(erase)
    cout = 3'b0;   // black
else if(v > 61)
    cout = 3'b110; // yellow
else
    cout = 3'b010; // green
```

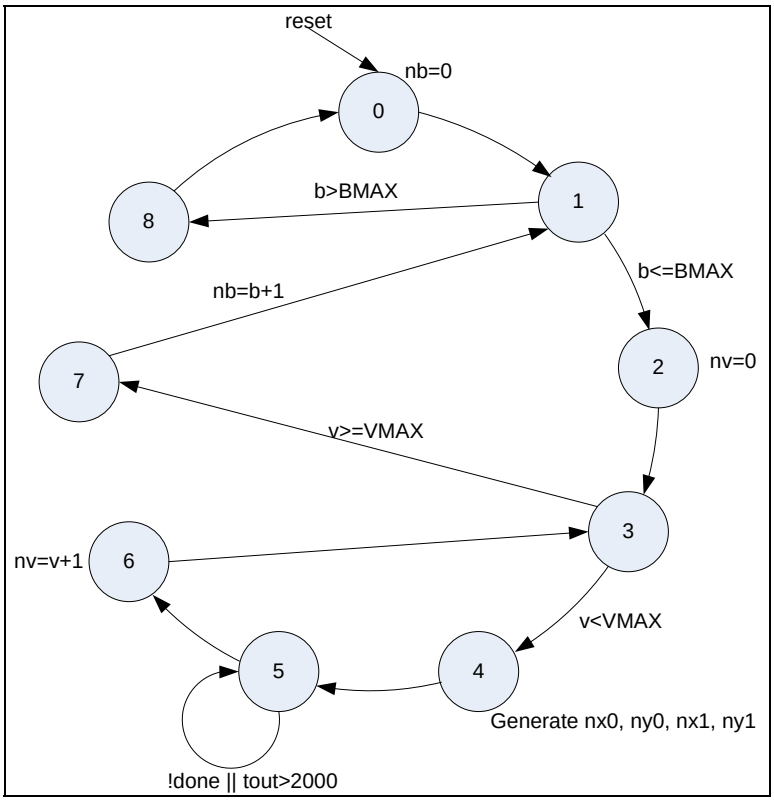


Figure 22 – Finite State Machine of *deng*

Table 1 – 3-Bit Display Color Codes [3]

c[2] - red	c[1] – green	c[0] – blue	Color
0	0	0	**Black**
0	0	1	**Blue**
0	1	0	**Green**
0	1	1	**Cyan**
1	0	0	**Red**
1	0	1	**Magenta**
1	1	0	**Yellow**
1	1	1	**White**

linedraw

There are seven signals which got from *deng*.

- deng_linedraw_start – start signal that let *linedraw* start to draw
- deng_linedraw_erase – this signal lets *linedraw* start to erase the screen.
- deng_linedraw_c – the color of the line.
- deng_linedraw_x0, deng_linedraw_y0
  the initial coordinates to draw the line.
- deng_linedraw_x1, deng_linedraw_y1
  the final coordinates of the line.

Also there is an input from *linedraw* which will be sent to *deng*.

- linedraw_deng_done
  this signal will active high when linedraw finish draw a line so deng can send the new line and let linedraw continue.

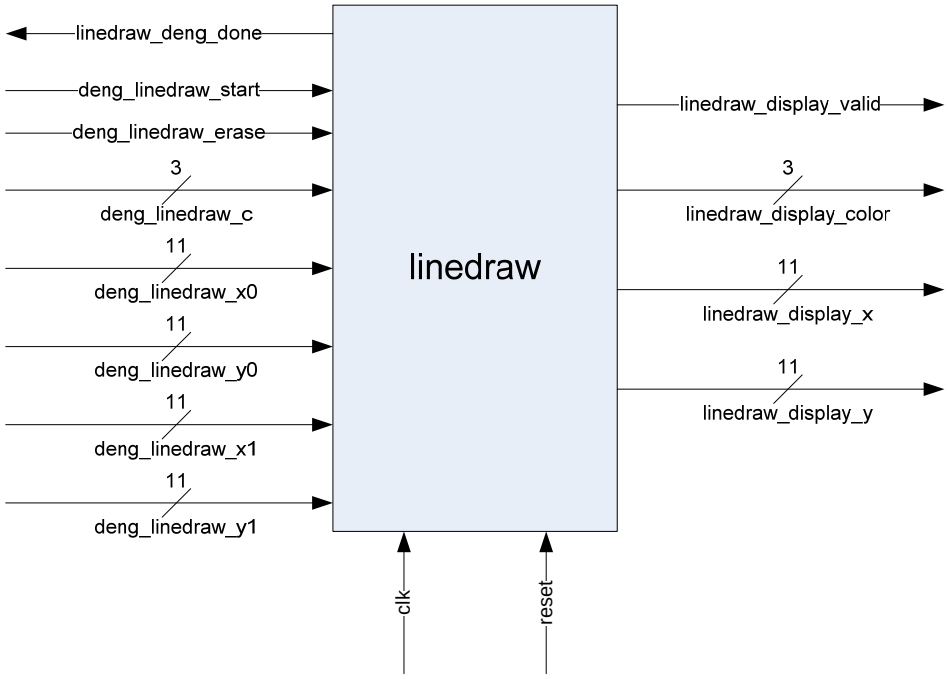


Figure 23 – *linedraw*

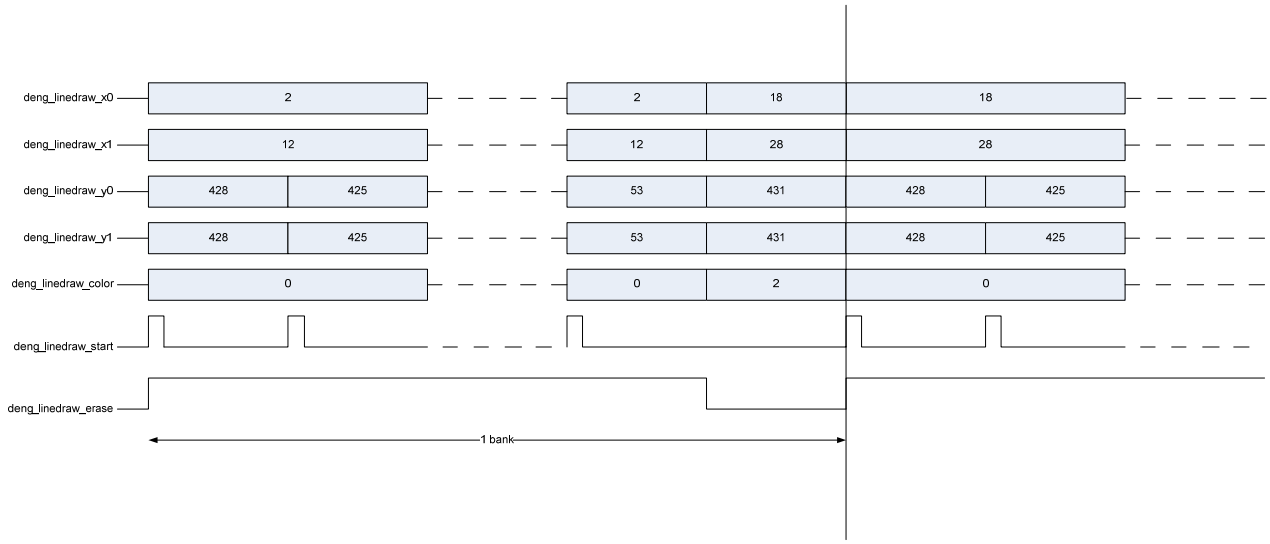


Figure 24 – functional specification of *deng_linedraw*

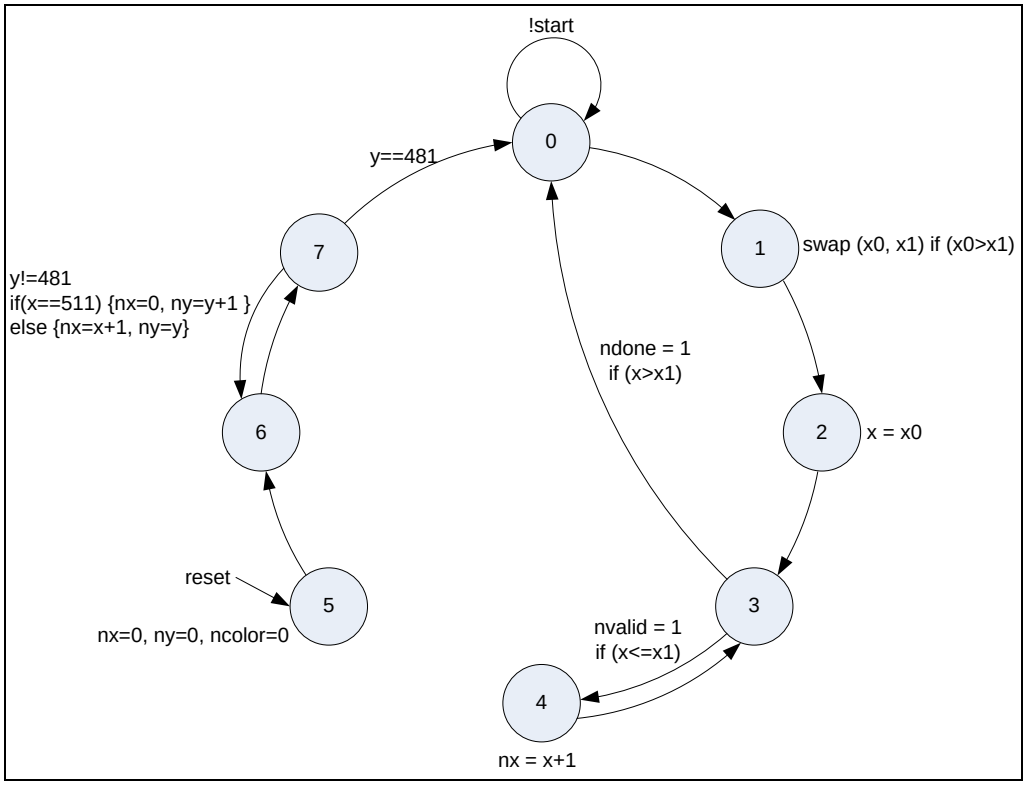


Figure 25 – Finite State Machine of *linedraw*

In *linedraw*, it generates each point to draw on a monitor according to the values which got from *deng*.
Here is algorithm of *linedraw*

```
// This is algorithm of linedraw 2008 02 24
// The display resolution is 512x480
// start = the signal from deng which will be asserted when the coordinates are generated
// (x0, y) = initial coordinate
// (x1, y) = final coordinate
// color = the color of the pixel.
// valid = it will be asserted when a pixel is sent to 'display' to plot on the monitor screen.
// done = it will be asserted if drawing a line is DONE then it will be sent back to deng so
linedraw can get a new coordinate.
//
// [coordinate] In linedraw, we got only 1's y since our drawing specification can be drawn only
horizontal line
// [color] According to deng, if the current data that just came in less than the previous, it wil
be removed.

while (start){
        initial the line (x0, x1, y, color){
             if (erase)
                    color = 0; //black
             else
                    color = deng_linedraw_color;
    }
        if (x0 > x1)
                swap (x0, x1)
        for (x=x0; x<=x1; x++){
                valid = 1;
                draw (x,y)
        }
        done = 1;
}

// clear screen
for (y=0; y==481; y++){
    for (x=0; x==511; x++){
         color = 0; // black
   }
}
```

display

There are four signals which got from *linedraw*.

- linedraw_display_valid
  This signal shows the validity of each coordinate that is sent in.
- linedraw_display_color
  This is the color of the coordinate that is sent in.
- linedraw_display_x
  x – coordinate which respects to 512x480 display
- linedraw_display_y
  y – coordinate which respects to 512x480 display

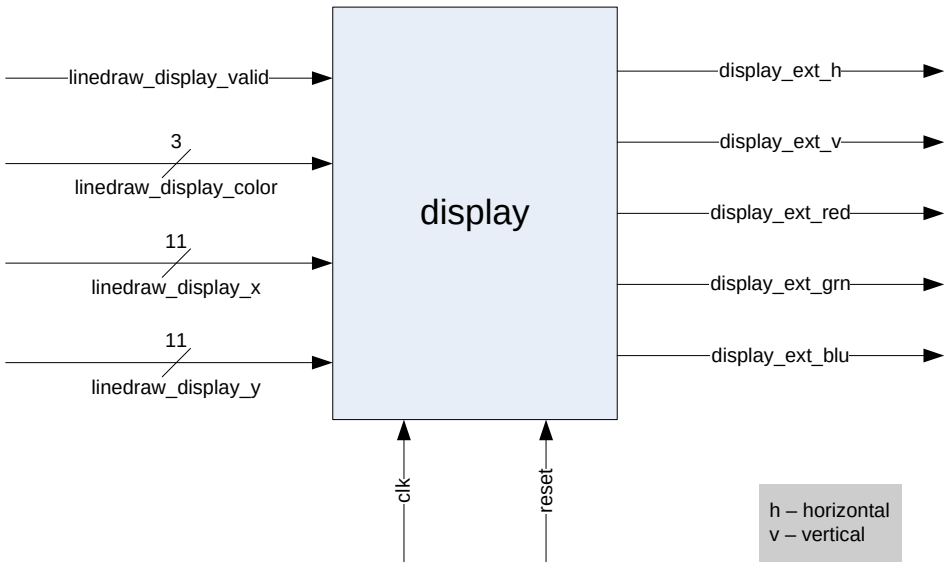


Figure 26 – *display*

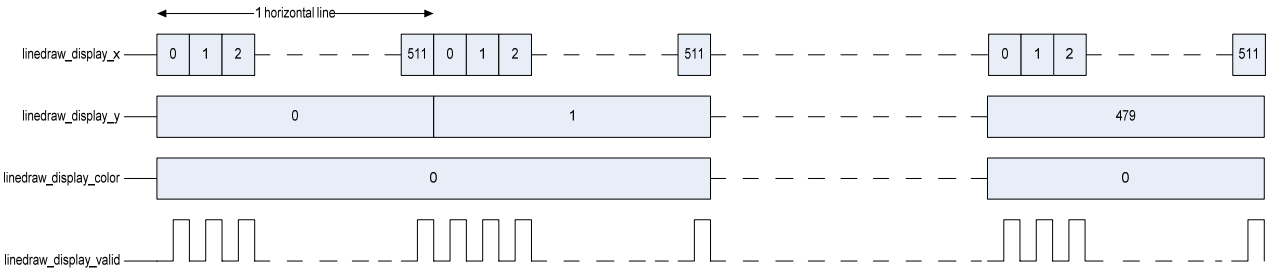


Figure 27 – functional specification of *linedraw_display*

In *display*, it changes the value of x, y and color into the proper format in order to draw on the VGA monitor with 512x480 resolutions.

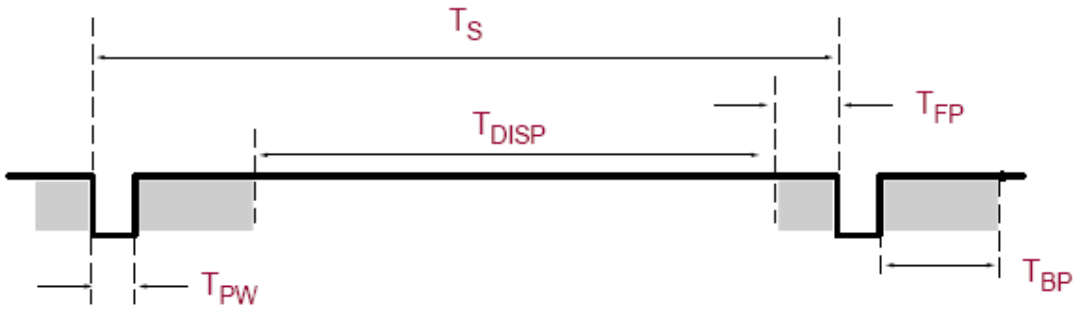


Figure 28 - VGA Control Timing [3]

VGA signal timing displays using a 25 MHz pixel clock and 60 Hz ±1 refresh for 640x480

Table 2 - 640x480 VGA Mode Timing [3]

Symbol	Parameter	Vertical Sync			Horizontal Sync		
		Time	Clocks	Lines	Time	Clocks (25MHz)	Clocks x2 (50MHz)
T_S	Sync pulse time	16.7 ms	416,800	521	32 µs	800	1,600
T_{DISP}	Display	15.36 ms	38,400	480	25.6 µs	(640) 512	(1, 280) 1,024
T_{PW}	Pulse width	64 µs	1,600	2	3.84 µs	96	192
T_{FP}	Front porch	320 µs	8,000	10	640 ns	16	32
T_{BP}	Back porch	928 µs	23,200	29	1.92 µs	48	96

Note: `PDELAY = 2

1. It generates horizontal and vertical position according to sync pulse time of each axis. So the range of horizontal position (hpos) is [0, 1599] – 800*2 clocks and for the range of vertical position (vpos) is [0, 520] – 521 lines.

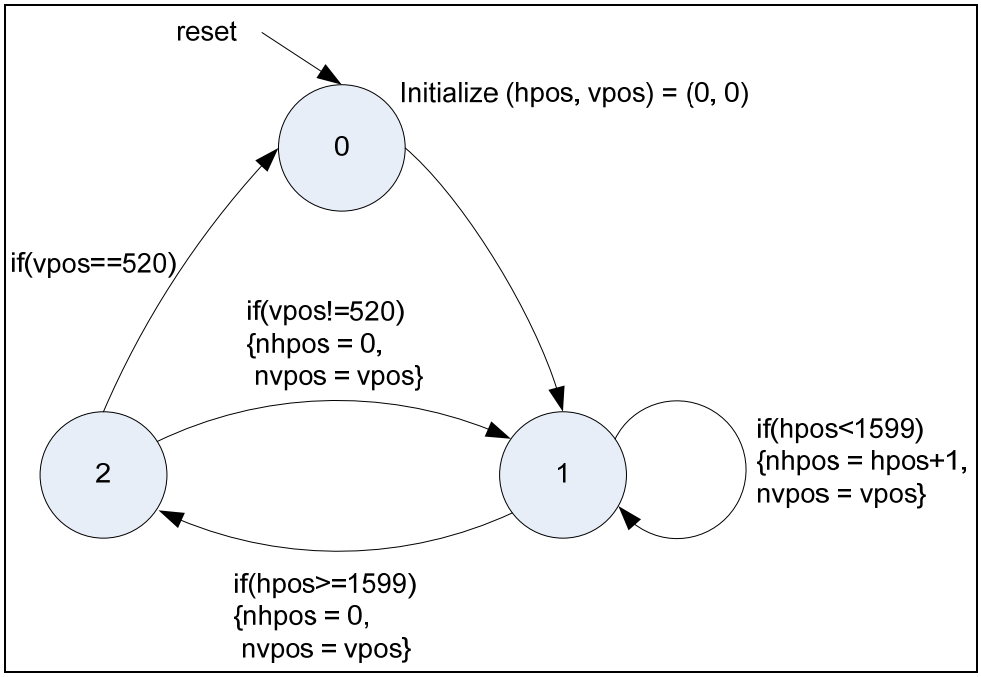
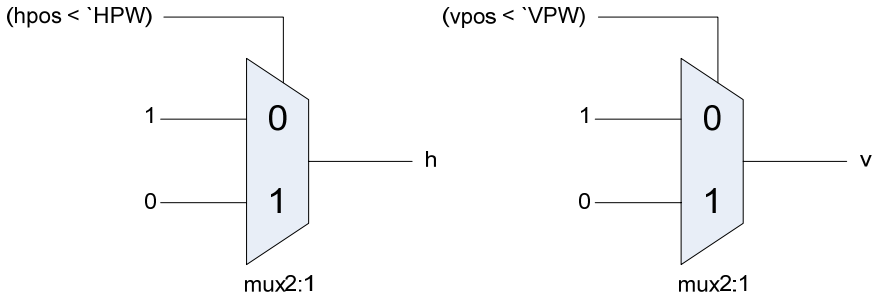


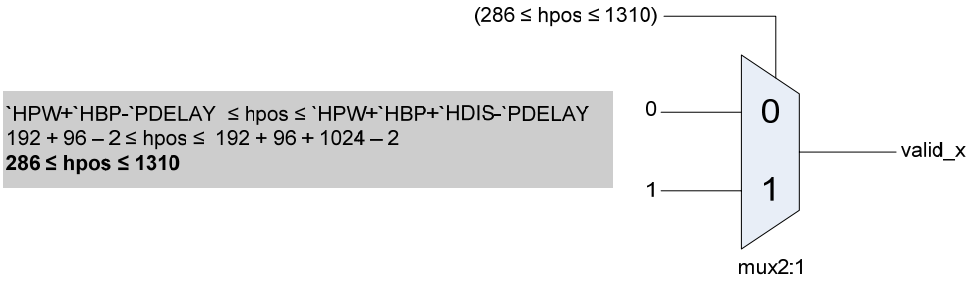
Figure 29 - Finite State Machine of *display*

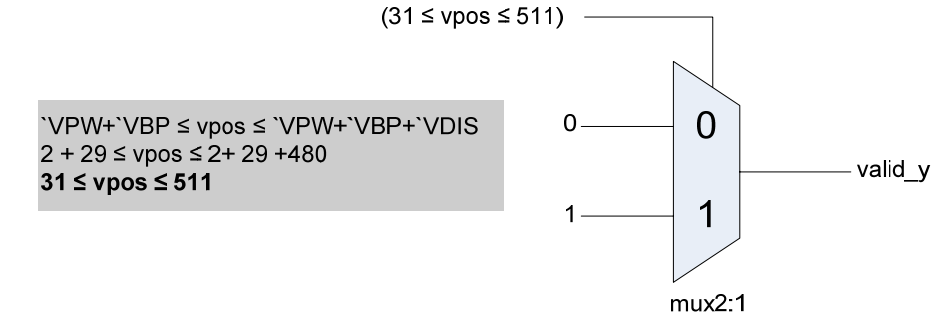
2. After getting the position of h and v, then it can find

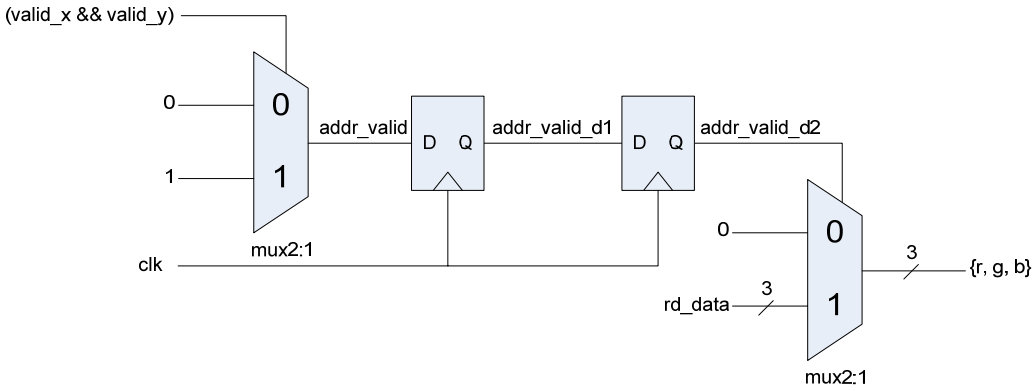
   - Horizontal sync (h) and Vertical sync (v)



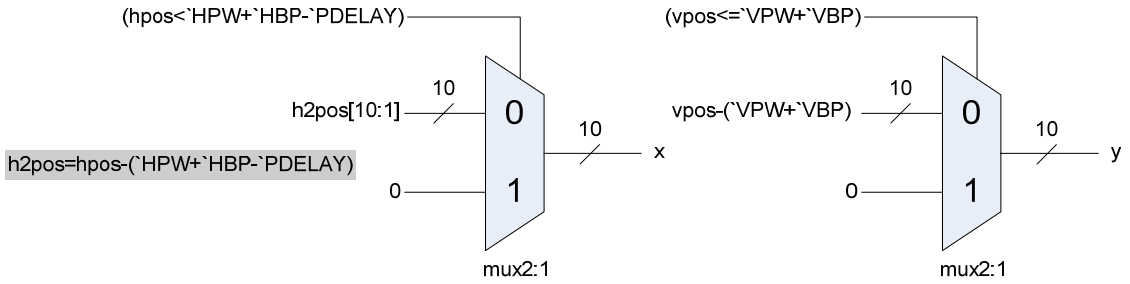
- Validity of display time (addr_valid) of horizontal sync (valid_x) and vertical sync (valid_y) to identify when each address is valid for getting the read data from frame buffer.



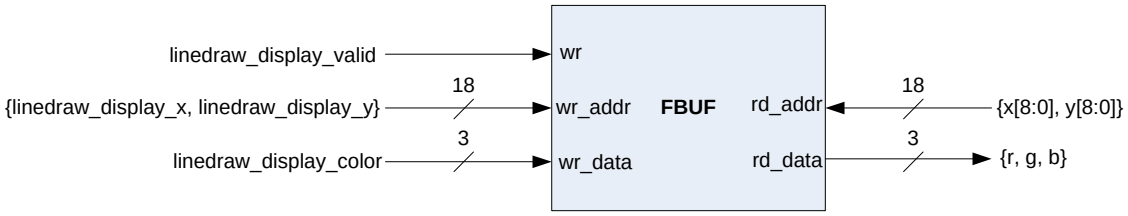




- Read address (rd_addr) so *display* can obtain read data (rd_data) which is the color from that address.
  Note: h2pos is the position when x is valid.



We do need frame buffer in *display* since VGA signal timing displays using a 25 MHz pixel clock. Regarding to frame buffer has a single port, it cannot read and write synchronously so we have to delay until the read data from frame buffer back to *display* for two clocks (`PDELAY = 2) i.e. one clock for read and one for write.



VGA (external)

There are five signals which got from *display*.

- display_ext_h – VGA signal for Horizontal Sync.
- display_ext_v – VGA signal for Vertical Sync.
- display_ext_red – VGA signal for red color.
- display_ext_grn – VGA signal for green color.
- display_ext_blu – VGA signal for blue color.

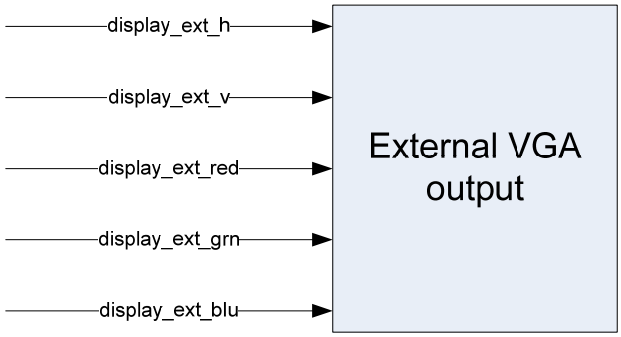


Figure 30 – External VGA output

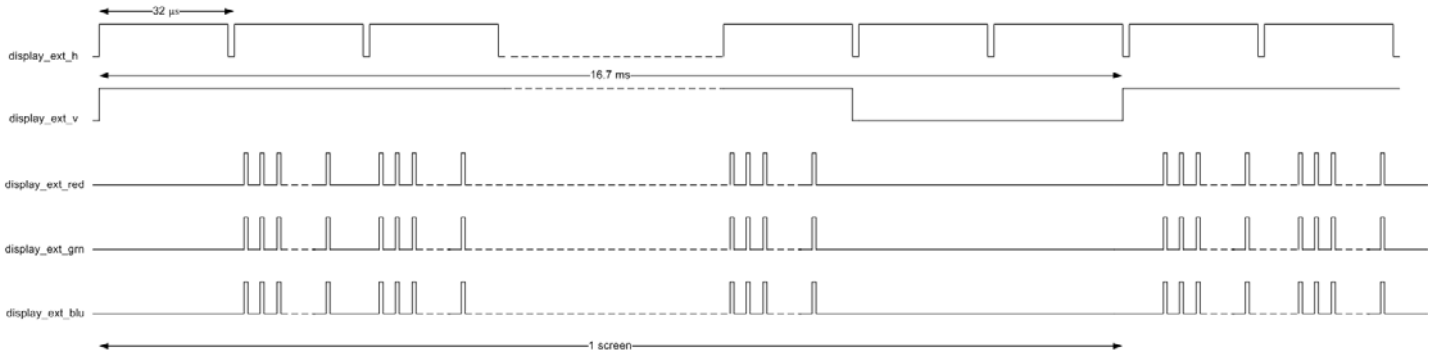


Figure 31 – functional specification of display_ext

The output will be shown on VGA monitor according to the input signals.

io_buffers (top module)

In top module, it will be empty according to the design guideline. It contains only module instantiation of every module.

Chapter 3: Fabrication, Construction

PCB (Printed Circuit Board)

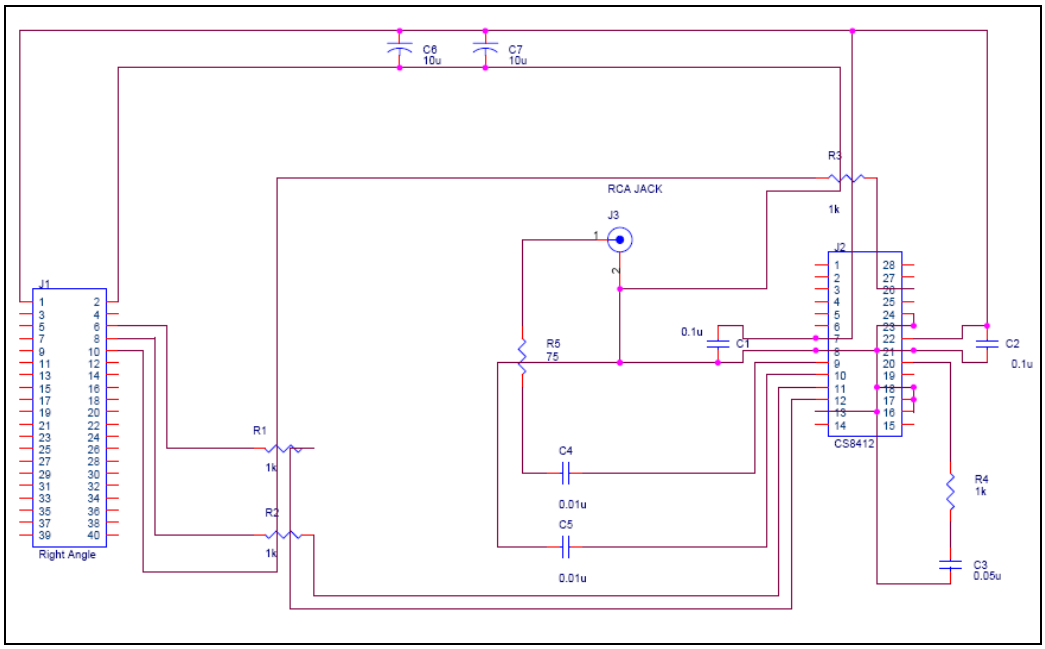


Figure 32 – Schematic for PCB

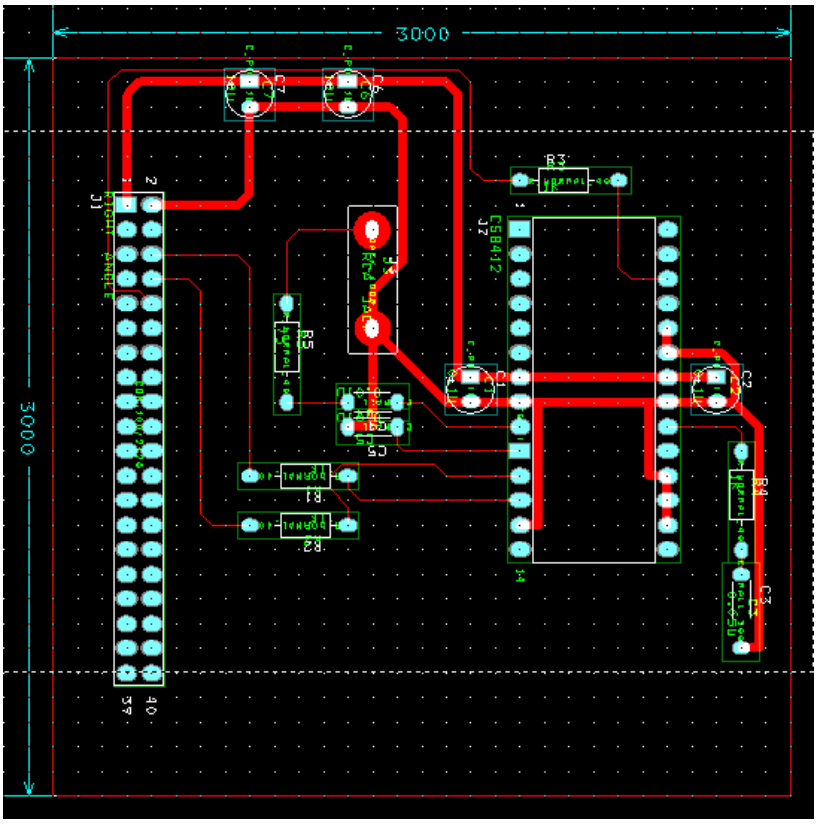


Figure 33 – Routed PCB

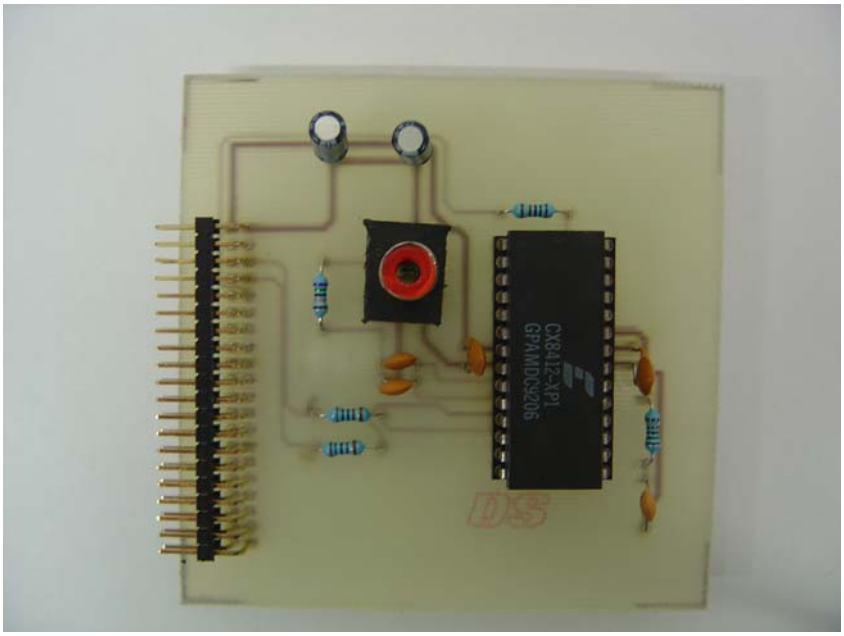


Figure 34 – Actual PCB

PCB Design Tools: OrCAD 9.1

Dimensions: 3x3 $inches^2$

Layer: 1 layer (bottom layer)

PCB components:

- CS8412
- Male pin header strip right angle 2x20 pins
- Female RCA jack
- Ceramic Capacitors
    - 2 x 0.01µF
    - 2 x 0.1 µF
    - 1 x 0.5 µF
- Electrolytic Capacitors
    - 2 x 10 µF
- Resistors
    - 4 x 1 kΩ
    - 1 x 75 Ω

Chapter 4: Test, Experiment, Equipment

For testing, we have three procedures of testing

1. **Unit testing**
   We did unit testing when each module is done. For each module, we fed the ideal inputs to that module and check whether or not the outputs which come out are same as the expected output. Thus the unit testing should satisfy the functional specification of the module.

2. **Integration testing**
   Our type of integration testing is bottom-up since it can be noticed easily whether or not it is working because it will display the result what we are testing on the screen. We did simulation starting from the lowest block to make sure that it can be placed in the real environment. And then we gradually integrate each block together until all blocks are added.

3. **System testing**
   After integration testing is successful, we assembled those modules together as a whole system then test it again to check whether all are working well or not.

Chapter 5: Conclusion and Recommendation

This report presented a brief overview of our project *Digital Spectrum Analyzer* starting from design, implementation to testing. *Digital Spectrum Analyzer* is the device that used to display the amplitude of each frequency from the CD player digital output. Actually this kind of software that can do similar to our project is already existed but our project is the specific hardware provides faster performance because it does not have to spend time on operating system and its size also smaller than the actual personal computer with lower cost.

The project also gives us the experiences in hardware design, project management and working as a team. We applied all knowledge what we have studied especially in Digital Logic Design and Digital Signal Processing. That makes us more confident to say that our knowledge that we have learnt is able to make use of it.

For the future contribution, we would like to improve this *Digital Spectrum Analyzer* can generate the output in various types for example, it can show the frequency domain on three dimensions (three axis) instead of two.

References

[1] CS8412 Datasheet

[Viewed at http://www.tranzistoare.ro/datasheets/700/158850_DS.pdf]

[2] http://pc.watch.impress.co.jp/docs/2004/0428/nics8412.jpg

[3] Spartan 3 Kit board User Guide

[Viewed at http://www.xilinx.com/support/documentation/boards_and_kits/ug130.pdf]

[4] http://members.optushome.com.au/jekent/Images/Spartan-3-Starter-Kit.gif

[5] http://www.graysonline.com.au/sale.asp?SALE_ID=18064

[-] CE3704 Digital Logic Design Lectures

[Viewed at http://www.talent.in.th/~ce3704]